Applications of Depth-First Search

Lecture Notes with Algorithms, Worked Examples and Graph Diagrams

Data Structures and Algorithms
Undergraduate and GATE Preparation

Contents

1 Introduction to Depth-First Search **3**
1.1 Basic DFS Algorithm . . . 3
1.2 Major Applications . . . 3

2 Connected Components Using DFS **3**
2.1 Definition . . . 3
2.2 Example Graph . . . 3
2.3 Step-by-Step DFS . . . 4
2.4 Algorithm . . . 4

3 Strongly Connected Components **5**
3.1 Definition . . . 5
3.2 Example Graph . . . 5
3.3 Pass 1: DFS on Original Graph . . . 5
3.4 Pass 2: DFS on Transpose Graph . . . 5
3.5 Kosaraju Algorithm . . . 6

4 Articulation Points Using DFS **7**
4.1 Definition . . . 7
4.2 Example Graph . . . 7
4.3 Removing Vertex C . . . 7
4.4 Removing Vertex D . . . 7
4.5 Discovery Time and Low Value . . . 7
4.6 Articulation Point Conditions . . . 8
4.7 Worked Numerical Example . . . 8
4.8 Algorithm . . . 9

5 Spanning Tree Using DFS **10**
5.1 Definition . . . 10
5.2 Example . . . 10
5.3 Spanning Forest . . . 11
5.4 Algorithm . . . 11

6 Biconnected Components Using DFS **12**
6.1 Definition . . . 12
6.2 Example Graph . . . 12
6.3 Identifying the Components . . . 12
6.4 How DFS Finds Biconnected Components . . . 12
6.5 Step-by-Step Example . . . 13
6.6 Algorithm . . . 14

7 Discovery and Finishing Times **15**
7.1 Definition . . . 15
7.2 Worked Example . . . 15
7.3 Step-by-Step Events . . . 16
7.4 Final Timestamp Table . . . 16
7.5 Important Properties . . . 16
7.6 Algorithm . . . 17

8 Comparison of DFS Applications **17**

9 Important Formulas **18**

10 Practice Numerical Questions **18**

11 Solutions to Practice Questions **19**

1. Introduction to Depth-First Search

Depth-First Search (DFS) is a graph traversal algorithm that explores a path as deeply as possible before backtracking to explore alternative paths.

DFS uses a stack, either explicitly or through recursion.

1.1 Basic DFS Algorithm

```
def dfs(u, adj, visited):
    visited.add(u)
    print(u, end=" ")

    for v in adj[u]:
        if v not in visited:
            dfs(v, adj, visited)
```

For a graph represented using adjacency lists:

$$\boxed{\text{Time Complexity} = O(V + E)}$$

The auxiliary space complexity is $O(V)$.

1.2 Major Applications

1. Connected components
2. Strongly connected components
3. Articulation points
4. Spanning trees
5. Biconnected components
6. Discovery and finishing times

2. Connected Components Using DFS

2.1 Definition

A **connected component** is a maximal connected subgraph of an undirected graph in which every pair of vertices is connected by a path.

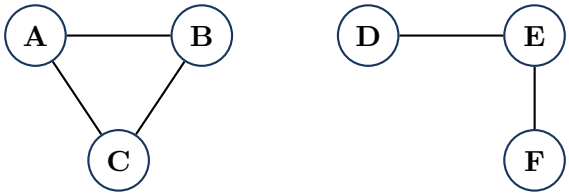
2.2 Example Graph

Consider an undirected graph with vertices

$$V = \{A, B, C, D, E, F\}$$

and edges

$$E = \{AB, BC, CA, DE, EF\}.$$



Component 1 Component 2

2.3 Step-by-Step DFS

1. Start DFS from vertex A.
2. Visit $A \to B \to C$.
3. All vertices of the first component have been visited.
4. Search for an unvisited vertex.
5. Vertex D is still unvisited.
6. Start another DFS: $D \to E \to F$.
7. All vertices have now been visited.

Therefore,

$$C_1 = \{A, B, C\}$$

$$C_2 = \{D, E, F\}.$$

$$\boxed{\text{Number of Connected Components} = 2}$$

2.4 Algorithm

```python
def connected_components(adj):
    visited = set()
    components = []

    def dfs(u, component):
        visited.add(u)
        component.append(u)

        for v in adj[u]:
            if v not in visited:
                dfs(v, component)

    for u in adj:
        if u not in visited:
            component = []
            dfs(u, component)
            components.append(component)

    return components
```

Key observation: Each new DFS call from an unvisited vertex identifies one connected component.

$$\boxed{T(V, E) = O(V + E)}$$

3. Strongly Connected Components

3.1 Definition

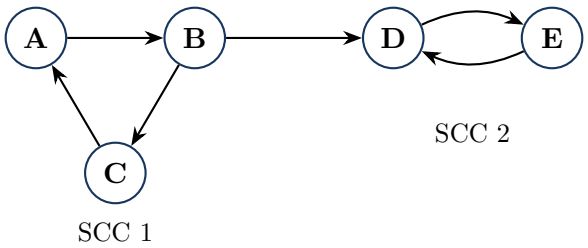
A **strongly connected component (SCC)** is a maximal set of vertices in a directed graph such that every vertex is reachable from every other vertex in the set.

Strong connectivity requires reachability in both directions.

3.2 Example Graph

Consider the directed graph

$$E = \{A \to B,\ B \to C,\ C \to A,\ B \to D,\ D \to E,\ E \to D\}.$$



We apply **Kosaraju's algorithm**.

3.3 Pass 1: DFS on Original Graph

Start DFS at A and examine outgoing neighbors in the order shown.

The discovery sequence is

$$A \to B \to C, \qquad B \to D \to E.$$

After recursive backtracking, the finishing sequence is

$$C,\ E,\ D,\ B,\ A.$$

Therefore, decreasing finishing-time order is

$$A,\ B,\ D,\ E,\ C.$$

3.4 Pass 2: DFS on Transpose Graph

Construct the transpose graph G^T by reversing all directed edges.

For every edge $u \to v$ in G, the transpose contains $v \to u$.

Run DFS on G^T in decreasing order of the original finishing times.

1. Start at A.
2. Reach vertices A, C, B.

3. First SCC is $\{A, B, C\}$.
4. Continue from the next unvisited vertex D.
5. Reach vertices D, E.
6. Second SCC is $\{D, E\}$.

$$\boxed{\text{SCC}_1 = \{A, B, C\}}$$

$$\boxed{\text{SCC}_2 = \{D, E\}}$$

3.5 Kosaraju Algorithm

```python
def kosaraju(adj):
    visited = set()
    order = []

    def dfs1(u):
        visited.add(u)

        for v in adj[u]:
            if v not in visited:
                dfs1(v)

        order.append(u)

    for u in adj:
        if u not in visited:
            dfs1(u)

    transpose = {u: [] for u in adj}

    for u in adj:
        for v in adj[u]:
            transpose[v].append(u)

    visited.clear()
    components = []

    def dfs2(u, component):
        visited.add(u)
        component.append(u)

        for v in transpose[u]:
            if v not in visited:
                dfs2(v, component)

    for u in reversed(order):
        if u not in visited:
            component = []
            dfs2(u, component)
            components.append(component)

    return components
```

The total complexity of Kosaraju's algorithm is

$$\boxed{O(V + E)}.$$

4. Articulation Points Using DFS

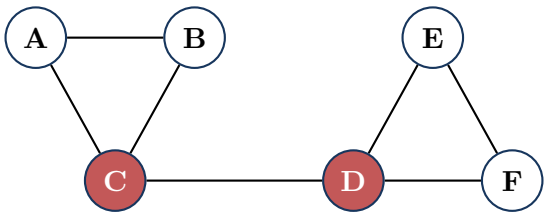
4.1 Definition

An **articulation point**, also called a cut vertex, is a vertex whose removal together with its incident edges increases the number of connected components of an undirected graph.

4.2 Example Graph

Consider the following graph:

$$E = \{AB, BC, CA, CD, DE, EF, FD\}.$$



4.3 Removing Vertex C

After removing C, the remaining graph has two connected components:

$$\{A, B\}, \qquad \{D, E, F\}.$$

Thus, C is an articulation point.

4.4 Removing Vertex D

After removing D, the graph splits into

$$\{A, B, C\}, \qquad \{E, F\}.$$

Thus, D is an articulation point.

$$\boxed{\text{Articulation Points} = \{C, D\}}$$

4.5 Discovery Time and Low Value

DFS maintains two important values.

Discovery time:

$$\text{disc}[u]$$

is the time at which vertex u is first discovered.

Low-link value:

$$\text{low}[u]$$

is the minimum discovery time reachable from u or a descendant of u through DFS-tree edges followed by at most one back edge.

Initially,

$$\text{low}[u] = \text{disc}[u].$$

For a DFS-tree edge (u, v),

$$\text{low}[u] = \min\left(\text{low}[u], \text{low}[v]\right).$$

For a back edge from u to ancestor w,

$$\text{low}[u] = \min\left(\text{low}[u], \text{disc}[w]\right).$$

4.6 Articulation Point Conditions

Condition 1: Root vertex
A DFS root is an articulation point if it has more than one DFS-tree child.

Condition 2: Non-root vertex
A non-root vertex u is an articulation point if there exists a DFS child v such that

$$\boxed{\text{low}[v] \geq \text{disc}[u]}$$

4.7 Worked Numerical Example

Perform DFS from A, visiting neighbors alphabetically.

The DFS discovery order is

$$A \to B \to C \to D \to E \to F.$$

The back edges are $C \to A$ and $F \to D$.

Vertex	Discovery	Low	Parent
A	1	1	–
B	2	1	A
C	3	1	B
D	4	4	C
E	5	4	D
F	6	4	E

For vertex C and its child D:

$$\text{low}[D] = 4$$

$$\text{disc}[C] = 3.$$

Since

$$4 \geq 3,$$

C is an articulation point.

For vertex D and its child E:

$$\text{low}[E] = 4$$

and

$$\text{disc}[D] = 4.$$

Therefore,

$$4 \geq 4.$$

Hence, D is an articulation point.

For vertex B and its child C:

$$\text{low}[C] = 1 < \text{disc}[B] = 2.$$

Hence, B is not an articulation point. The back edge $C \to A$ provides an alternate connection.

4.8 Algorithm

```
def articulation_points(adj):
    visited = set()
    disc = {}
    low = {}
    parent = {}
    points = set()
    timer = 0

    def dfs(u):
        nonlocal timer

        visited.add(u)
        timer += 1
        disc[u] = low[u] = timer
        children = 0

        for v in adj[u]:

            if v not in visited:
                parent[v] = u
                children += 1

                dfs(v)

                low[u] = min(
                    low[u], low[v]
                )
```

```
                if (u not in parent
                        and children > 1):
                    points.add(u)

                if (u in parent
                        and low[v] >= disc[u]):
                    points.add(u)

            elif parent.get(u) != v:
                low[u] = min(
                    low[u], disc[v]
                )

    for u in adj:
        if u not in visited:
            dfs(u)

    return points, disc, low
```

5. Spanning Tree Using DFS

5.1 Definition

A **spanning tree** of a connected undirected graph is an acyclic connected subgraph containing all vertices of the original graph.
A spanning tree of a graph with V vertices contains exactly $V - 1$ edges.

5.2 Example

Consider the graph

$$E = \{AB, BC, CA, CD, DE, EF, FD\}.$$

Perform DFS starting at A, visiting neighbors alphabetically.

The DFS discovery order is

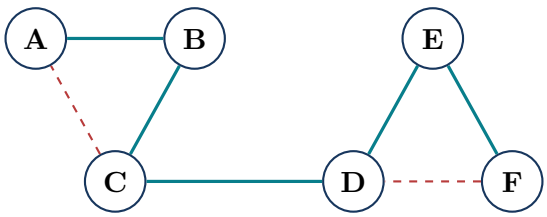
$$A \to B \to C \to D \to E \to F.$$

The DFS tree edges are

$$\{AB, BC, CD, DE, EF\}.$$

The non-tree edges are

$$\{CA, FD\}.$$



Solid teal edges: DFS tree edges.

Dashed red edges: Non-tree edges.

The graph has

$$V = 6.$$

Therefore,

$$V - 1 = 6 - 1 = 5.$$

$$\boxed{\text{DFS Spanning Tree Edges} = 5}$$

5.3 Spanning Forest

For a disconnected undirected graph, DFS generates one spanning tree for each connected component.

If the graph contains V vertices and C connected components,

$$\boxed{\text{DFS Forest Edges} = V - C}$$

For example, if

$$V = 20, \qquad C = 4,$$

then

$$20 - 4 = 16$$

DFS forest edges are produced.

5.4 Algorithm

```
def dfs_spanning_forest(adj):
    visited = set()
    tree_edges = []

    def dfs(u):
        visited.add(u)

        for v in adj[u]:
            if v not in visited:
                tree_edges.append((u, v))
                dfs(v)

    for u in adj:
        if u not in visited:
            dfs(u)

    return tree_edges
```

6. Biconnected Components Using DFS

6.1 Definition

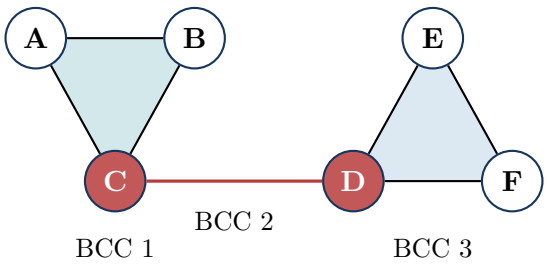
A **vertex-biconnected component**, also called a block, is a maximal connected subgraph with no articulation point within that subgraph.
Two distinct blocks may share articulation vertices, but cannot share edges.

Under the standard block decomposition, a bridge with its two endpoints forms a separate biconnected component.

6.2 Example Graph

Again consider

$$E = \{AB, BC, CA, CD, DE, EF, FD\}.$$



6.3 Identifying the Components

The first triangle forms

$$\text{BCC}_1 = \{A, B, C\}.$$

The bridge CD forms

$$\text{BCC}_2 = \{C, D\}.$$

The second triangle forms

$$\text{BCC}_3 = \{D, E, F\}.$$

$$\boxed{\text{Number of Biconnected Components} = 3}$$

6.4 How DFS Finds Biconnected Components

DFS maintains:

1. Discovery time $\text{disc}[u]$
2. Low-link value $\text{low}[u]$

3. A stack of DFS edges

Whenever a DFS-tree edge (u, v) satisfies

$$\boxed{\text{low}[v] \geq \text{disc}[u]}$$

pop edges from the stack until (u, v) is popped.

The popped edges form one biconnected component.

6.5 Step-by-Step Example

The DFS traversal is

$$A \to B \to C \to D \to E \to F.$$

Step 1:

The back edge $F \to D$ closes the cycle $D - E - F - D$.

Since

$$\text{low}[E] = 4 \geq \text{disc}[D] = 4,$$

pop edges

$$FD,\ EF,\ DE.$$

This forms

$$\text{BCC}_3 = \{D, E, F\}.$$

Step 2:

For the tree edge CD,

$$\text{low}[D] = 4 \geq \text{disc}[C] = 3.$$

Pop edge CD.

This forms

$$\text{BCC}_2 = \{C, D\}.$$

Step 3:

For the tree edge AB,

$$\text{low}[B] = 1 \geq \text{disc}[A] = 1.$$

Pop edges

$$CA,\ BC,\ AB.$$

This forms

$$\text{BCC}_1 = \{A, B, C\}.$$

6.6 Algorithm

```python
def biconnected_components(adj):
    disc = {}
    low = {}
    parent = {}
    stack = []
    components = []
    timer = 0

    def dfs(u):
        nonlocal timer

        timer += 1
        disc[u] = low[u] = timer

        for v in adj[u]:

            if v not in disc:
                parent[v] = u
                stack.append((u, v))

                dfs(v)

                low[u] = min(
                    low[u], low[v]
                )

                if low[v] >= disc[u]:
                    component = []

                    while stack:
                        edge = stack.pop()
                        component.append(edge)

                        if edge == (u, v):
                            break

                    components.append(
                        component
                    )

            elif (v != parent.get(u)
                  and disc[v] < disc[u]):

                low[u] = min(
                    low[u], disc[v]
                )
                stack.append((u, v))

    for u in adj:
        if u not in disc:
            dfs(u)

            if stack:
                components.append(
                    stack[:]
                )
                stack.clear()

    return components
```

The algorithm runs in

$$\boxed{O(V+E)}$$

time for a simple undirected graph.

7. Discovery and Finishing Times

7.1 Definition

During DFS, every vertex receives two timestamps.

Discovery time $d[u]$:
The time at which vertex u is first visited.

Finishing time $f[u]$:
The time when DFS completes processing all adjacent vertices of u and returns from its recursive call.

The timer is incremented at every discovery and finishing event.

For every vertex,

$$\boxed{d[u] < f[u]}.$$

7.2 Worked Example

Consider the same graph

$$E = \{AB, BC, CA, CD, DE, EF, FD\}.$$

Start DFS at A and visit neighbors alphabetically.

The discovery order is

$$A \to B \to C \to D \to E \to F.$$

7.3 Step-by-Step Events

Time	Operation	Explanation
1	Discover A	Start DFS
2	Discover B	A to B
3	Discover C	B to C
4	Discover D	C to D
5	Discover E	D to E
6	Discover F	E to F
7	Finish F	Backtrack to E
8	Finish E	Backtrack to D
9	Finish D	Backtrack to C
10	Finish C	Backtrack to B
11	Finish B	Backtrack to A
12	Finish A	DFS completed

7.4 Final Timestamp Table

Vertex	Discovery $d[u]$	Finishing $f[u]$
A	1	12
B	2	11
C	3	10
D	4	9
E	5	8
F	6	7

7.5 Important Properties

Property 1: Parenthesis Theorem

For any two vertices of a DFS forest, the intervals

$$[d[u], f[u]]$$

and

$$[d[v], f[v]]$$

are either disjoint or one is contained entirely within the other.

For example,

$$d[A] < d[B] < f[B] < f[A].$$

Therefore, B is a descendant of A in the DFS tree.

Property 2: Topological Sorting

For a directed acyclic graph (DAG), vertices arranged in decreasing finishing time form a valid topological ordering.

Property 3: Strongly Connected Components

Kosaraju's algorithm uses decreasing finishing times to determine the traversal order for its second DFS pass.

7.6 Algorithm

```python
def dfs_timestamps(adj):
    visited = set()
    discovery = {}
    finishing = {}
    timer = 0

    def dfs(u):
        nonlocal timer

        visited.add(u)
        timer += 1
        discovery[u] = timer

        for v in adj[u]:
            if v not in visited:
                dfs(v)

        timer += 1
        finishing[u] = timer

    for u in adj:
        if u not in visited:
            dfs(u)

    return discovery, finishing
```

8. Comparison of DFS Applications

Application	Graph Type	Main Technique	Complexity
Connected components	Undirected	Repeated DFS from unvisited vertices	$O(V + E)$
Strongly connected components	Directed	Kosaraju: DFS, transpose, DFS	$O(V + E)$
Articulation points	Undirected	Discovery and low-link values	$O(V + E)$
Spanning tree	Connected undirected	Record DFS tree edges	$O(V + E)$
Biconnected components	Undirected	Low-link values and edge stack	$O(V + E)$
Discovery and finishing times	Both	Global DFS timer	$O(V + E)$

9. Important Formulas

1. DFS Forest Edges
For an undirected graph with V vertices and C connected components,

$$\boxed{V - C}$$

2. Non-root Articulation Point
For a DFS child v of non-root u,

$$\boxed{\text{low}[v] \geq \text{disc}[u]}$$

3. Bridge
For a DFS-tree edge (u, v),

$$\boxed{\text{low}[v] > \text{disc}[u]}$$

4. Biconnected Component Boundary

$$\boxed{\text{low}[v] \geq \text{disc}[u]}$$

Pop stack edges through (u, v).
5. Discovery and Finishing Times

$$\boxed{d[u] < f[u]}$$

10. Practice Numerical Questions

Q1. An undirected graph contains 15 vertices and 3 connected components. Determine the number of DFS forest edges.

Q2. A DFS tree edge (u, v) satisfies

$$\text{disc}[u] = 4, \qquad \text{low}[v] = 4.$$

Can u be an articulation point? Is the edge necessarily a bridge?

Q3. Explain why two DFS passes are used in Kosaraju's algorithm.

Q4. A connected graph has 10 vertices and 15 edges. Find the number of non-tree edges in a DFS spanning tree.

Q5. Explain the difference between an articulation point and a bridge.

Q6. Consider a DFS tree in which u is the parent of v and

$$\text{disc}[u] = 3, \qquad \text{low}[v] = 2.$$

Is u necessarily an articulation point because of the subtree rooted at v?

Q7. Explain why decreasing finishing time gives a topological ordering only when the directed graph is acyclic.

11. Solutions to Practice Questions

Solution 1

$$V - C = 15 - 3 = \boxed{12}.$$

Solution 2

Since

$$\text{low}[v] = 4 \geq \text{disc}[u] = 4,$$

u satisfies the articulation-point condition for this child if u is not the DFS root.

The edge is not necessarily a bridge because a bridge requires strict inequality:

$$\text{low}[v] > \text{disc}[u].$$

Solution 3

The first DFS determines finishing times. The second DFS, performed on the transpose graph in decreasing finishing-time order, extracts individual SCCs without merging different SCCs.

Solution 4

A spanning tree contains

$$V - 1 = 10 - 1 = 9$$

edges. Hence, the number of non-tree edges is

$$15 - 9 = \boxed{6}.$$

Solution 5

An articulation point is a vertex whose removal increases the number of connected components.

A bridge is an edge whose removal increases the number of connected components.

Solution 6

No, because

$$\text{low}[v] = 2 < \text{disc}[u] = 3.$$

The subtree rooted at v has an alternate connection to an ancestor of u.

Solution 7

In a DAG, DFS finishing times respect the direction of edges when vertices are placed in decreasing finishing-time order.

A directed cycle makes a topological ordering impossible, because no linear ordering can place every vertex before all its outgoing neighbors in a directed cycle.